

ISP Network Planner — Plan of Action (2-Person Build)

Course: CS3101 Design & Analysis of Algorithms **Team:** Person A, Person B (technical pipeline)

1. What the tool does

A tool that plans a mobile/internet network for a real city (Pune). Given customer locations and candidate tower sites, it decides which towers to build, how to wire them together, and how to deliver bandwidth — with each stage built on a different algorithm from the syllabus.

Stage	What it does	Algorithm	Syllabus unit
0	Load real road network, demand points, candidate tower sites	Graph build, DFS/BFS	Unit I
1	Pick cheapest tower set covering every customer (headline)	Backtracking (exact) vs Greedy (approximate) facility location	Unit IV + VI
2	Choose equipment tier per tower under budget	0/1 Knapsack (DP)	Unit III
3	Wire towers together with least cable	Prim's / Kruskal's MST	Unit II / IV
4	Push bandwidth from exchange to towers; find the bottleneck	Ford-Fulkerson max-flow, min-cut	Unit V
5	Route signal from tower to customer	Dijkstra / A*	Unit II
6	Dashboard showing the finished plan	—	Result

Why stage 1 is the headline: facility location is NP-hard. Backtracking finds the true optimum but stalls past ~15–20 candidate sites (2^n growth); greedy is instant but sometimes costlier. Demonstrating that trade-off live, with real numbers, is the demo payoff and the Unit VI tie-in for the viva.

2. Module split

Person A — Core algorithms (the graded centerpiece)

- Stage 0: pull the Pune road graph (OSMnx), generate candidate sites + demand points, verify connectivity (BFS/DFS)

- Stage 1: hand-written greedy + backtracking facility location, with branch-and-bound pruning; comparison chart (cost, runtime, coverage) as N scales
- Stage 2: 0/1 knapsack DP for equipment-tier selection per built tower

Person B — Network layer + presentation

- Stage 3: Prim's or Kruskal's MST connecting the towers A's module builds
- Stage 4: Ford–Fulkerson max-flow from exchange to towers; report the min-cut bottleneck
- Stage 5: Dijkstra shortest path from a tower to a customer on the real road graph
- Stage 6: Streamlit + Folium dashboard integrating A's and B's outputs onto the live map

3. The one rule that prevents integration failure

Before either of you writes stage code, agree on the shared data format — in writing, today.

Minimum interface to lock:

- **Built-tower object:** `{id, position (lat/lon or projected x/y), tier, build_cost}`
- **Graph object:** whatever OSMnx/NetworkX structure A builds in stage 0 — B's MST, flow, and Dijkstra all read from this same object, not a reconstruction of it.
- **Demand object:** `{id, position, weight}`

Person B cannot start stage 3 without at least a rough (even dummy) list of built towers from Person A. So:

Sequencing: Person A builds a minimal, even ugly, end-to-end stage 1 first — real algorithm, dummy/small data if needed — and hands off the tower list and graph object early. Polish comes after the handoff, not before. Person B can stub stage-1 output manually in the meantime to start stage 3 without waiting idle.

4. Milestones

Each has a **Done** = definition. Sync every 2-3 days minimum to catch format drift early.

#	Milestone	Owner	Done =
0	Data setup	A	Pune district graph loads via OSMnx; ~15-20 candidate sites + demand points plotted on a Folium map
1	Stage 1 (headline)	A	Greedy + backtracking both run on the real graph; comparison chart (cost/runtime) renders; interface handed to B
2	Stage 2 (DP tiers)	A	Knapsack DP assigns equipment tier per tower under budget
3	Stage 3 (MST)	B	Backbone connects all of A's built towers at minimum cable cost
4	Stage 4 (flow)	B	Max-flow computed from exchange to towers; bottleneck link reported
5	Stage 5 (routing)	B	Shortest path from a tower to a customer renders on the map
6	Stage 6 (dashboard)	B (integrates A's outputs)	One dashboard shows towers, coverage, backbone, flow, and the stage-1 comparison chart together
7	Theory + report + benchmarks	Both	NP-hardness writeup, runtime curves (greedy vs backtracking as N grows), final report

5. Tech stack

Purpose	Tool
Language	Python
Road network data	OSMnx
Graph handling	NetworkX
Stage 1 (graded core)	Hand-written greedy + backtracking — no library
MST / max-flow / Dijkstra	Hand-written preferred; NetworkX built-ins as fallback if time-constrained (still explain the theory in the report)
Dashboard	Streamlit + Folium
Comparison charts	matplotlib

6. Biggest risks

1. **Format drift between A and B.** Solved by locking the interface before coding (Section 3). Check it's still being honored at every sync.
 2. **Stage 1 delay blocks everything downstream.** A should prioritize a rough end-to-end version over a polished one, and hand off early.
 3. **Backtracking without pruning won't scale even to 15 sites.** Branch-and-bound pruning isn't optional — it's required for both correctness-at-scale and for the demo to run live without hanging.
 4. **Coordinate projection errors** (lat/lon vs. metric) silently produce wrong coverage radii. Sanity-check with a plot before trusting any stage-1 number.
-

7. Immediate next action

Person A starts milestone 0 today — pull the Pune graph, generate sites, confirm the map renders. Person B can review the module split and prep stage-3/4 logic against dummy tower data while waiting on A's handoff.